

Laboratorio 3 – MQTT hacia Azure IoT Central

Protocolo visible, mediciones y comparación con el SDK

Escenario aplicado: UNAB-Ambiental

José Alejandro Téllez Prada – U00060479

Docente: Javier Pinzón

Bucaramanga, Santander

Septiembre de 2026

1. Objetivos cumplidos

- Explicar qué protocolo usa realmente un dispositivo hacia IoT Central: MQTT 3.1.1 sobre TLS (puerto 8883), autenticación SAS (HMAC-SHA256), pasando primero por DPS y luego por el IoT Hub subyacente de la app.
- Publicar las mismas 3 variables del Lab 2 (*Temperature*, *Humidity*, *Illuminance*) por dos caminos: SDK Python (`python-vm-01.py`) y cliente MQTT explícito (`mqtt_explicito.py`, paho puro, sin `azure-iot-device`).
- Mantener el ESP32 de Wokwi enviando por MQTT/TLS a la misma aplicación.
- Medir tamaño de payload, intervalo, latencia y comportamiento ante corte y ante QoS 0/1/2.

2. Flujo real: dispositivo → DPS → IoT Hub → IoT Central

El siguiente diagrama resume el flujo capturado directamente de los logs (`evidencias/miapp-2026-09-13/dps-manu` reconstruido a mano con MQTT puro (sin SDK):

Detalles verificados en la Etapa 1 y confirmados de nuevo al ejecutar el cliente explícito: el subdominio `{id-scope}.global...` no resuelve (hay que conectar a `global.azure-devices-provisioning.net` a secas y llevar el *id scope* en el username); tras un `assigning` corresponde un GET de estado de operación, no un re-PUT (los re-PUT generan `409 / 409203 Precondition failed`).

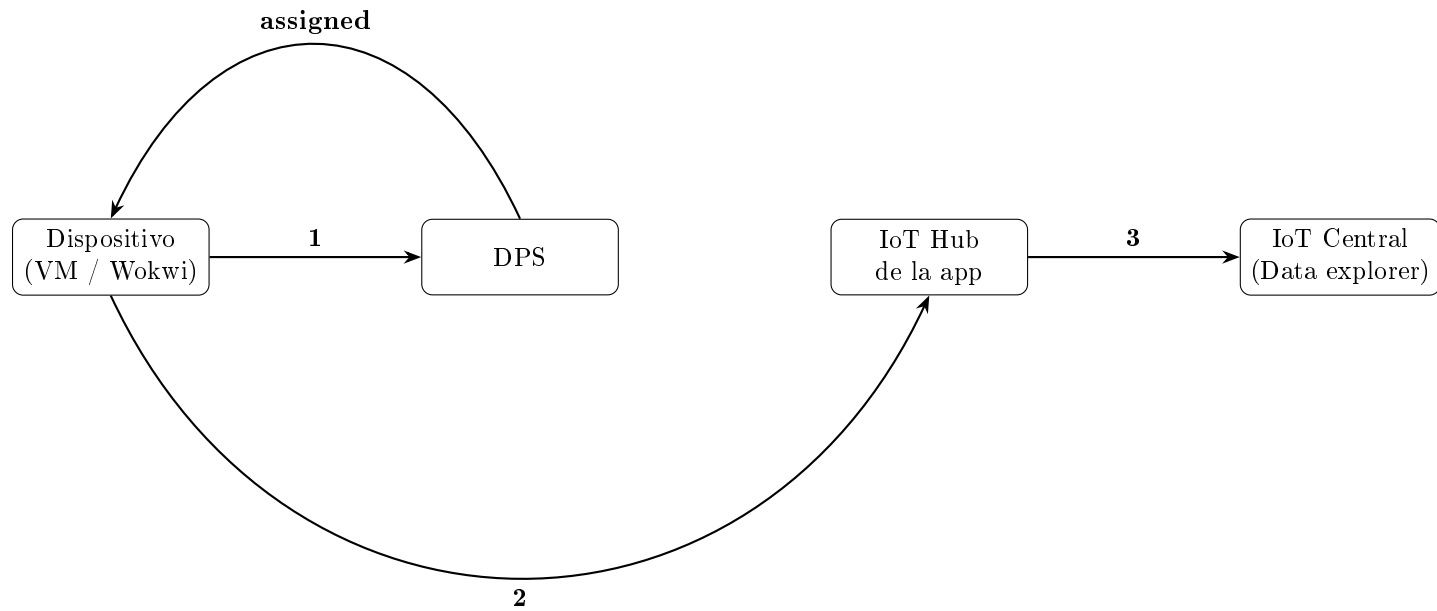


Figura 1: Flujo MQTT → DPS → IoT Hub → IoT Central, verificado a mano con `mqtt_explicito.py`. **1:** PUT register (TLS 8883, SAS) → 202 assigning → GET operationstatus. **Assigned:** 200 assigned, devuelve el `assignedHub`. **2:** CONNECT TLS 8883 al hub asignado (username=hub/device, password=SAS). **3:** PUBLISH telemetría → PUBACK (QoS 1) → ingestión en Data explorer.

3. Tabla de mediciones (cliente explícito, device 23z8rpgm6s4)

Concepto	Valor medido	Evidencia
Tamaño payload (3 variables)	60 bytes {"Temperature":..., "Humidity":..., "Illuminance":...} (1 decimal)	UTF-8 qos1.jsonl
Topic de telemetría	devices/23z8rpgm6s4/messages/events/	todos los logs
Intervalo configurado vs. observado	10 s vs. 10.0–10.1 s	qos1.jsonl
Latencia publish → PUBACK (QoS 1)	avg 230.6 ms (mín 206.1, máx 333.9; n=6)	qos1.log SUMMARY
Latencia publish → ack (QoS 0)	0.3–0.4 ms = ack local, sin confirmación del broker	qos0.log
Corte de red a los 25 s (QoS 1)	socket cerrado → reconexión automática con backoff (2 conexiones) → 6/6 acks, 0 mensajes perdidos	corte.log SUMMARY
QoS 0	aceptado por Central (publica sin confirmación)	qos0.log
QoS 1	aceptado; handshake PUBLISH → PUBACK normal	qos1.log
QoS 2	PUBLISH QoS 2 aceptado, pero el broker responde PUBACK (6/6) y nunca envía PUBREC/PUBREL/PUBCOMP (0/6) → degradado a QoS 1 en el alambre	qos2.log (-debug-packets)

4. Comparación de los tres caminos (misma app, mismas 3 variables)

Criterio	SDK Python (Lab 2)	MQTT explícito (paho)	Wokwi ESP32 (Arduino)
Facilidad	Alta (2 llamadas)	Media (SAS/topics/subscribe a mano)	Baja-media (SDK C + estado a mano)
Control/visibilidad del protocolo	Nula (es la “caja negra”)	Total (paquetes, QoS, topics)	Parcial (logs del sketch)
Payload	60 B	60 B (idéntico, misma plantilla)	60 B
Latencia publish→ack	no medía nada	230.6 ms avg medida	depende del navegador/Wokwi
Reconexión	automática del SDK	automática de paho (probada con corte)	manual: <code>WiFi.reconnect()</code> + máquina de estados
Utilidad para UNAB-Ambiental	operación diaria del nodo VM	diagnóstico (¿TLS? ¿SAS? ¿topic? ¿QoS?)	único camino en hardware real

Conclusión de QoS

IoT Central/IoT Hub admite QoS 0 y QoS 1, y **no implementa QoS 2**: al pedirlo, la telemetría no falla, pero el broker confirma con un simple PUBACK – es decir, la promesa MQTT de “exactamente

una vez” no existe contra Central. Elegir QoS 1 da “al menos una vez” con menor tráfico, y es justamente lo que el SDK ya hace por defecto. Por eso el QoS 2 medido muestra un RTT casi igual al de QoS 1 (252.8 ms vs. 230.6 ms): el paso extra del handshake simplemente nunca ocurre.

¿Cuándo usar el SDK y cuándo un cliente MQTT desnudo?

El SDK sigue siendo la opción correcta para el nodo operativo del salón: menos código y reconexión ya probada. El cliente explícito queda como **herramienta de diagnóstico**: cuando Central no reciba datos, `mqtt_explicito.py -debug-packets` permite distinguir en minutos si el problema es TLS, SAS, el topic o el QoS – algo que no se puede observar desde el SDK. Y si el proyecto migrara algún día a un broker propio (por ejemplo Mosquitto en el salón), el cliente explícito es reutilizable casi sin cambios, mientras que el código basado en el SDK tendría que reescribirse.

5. Evidencias

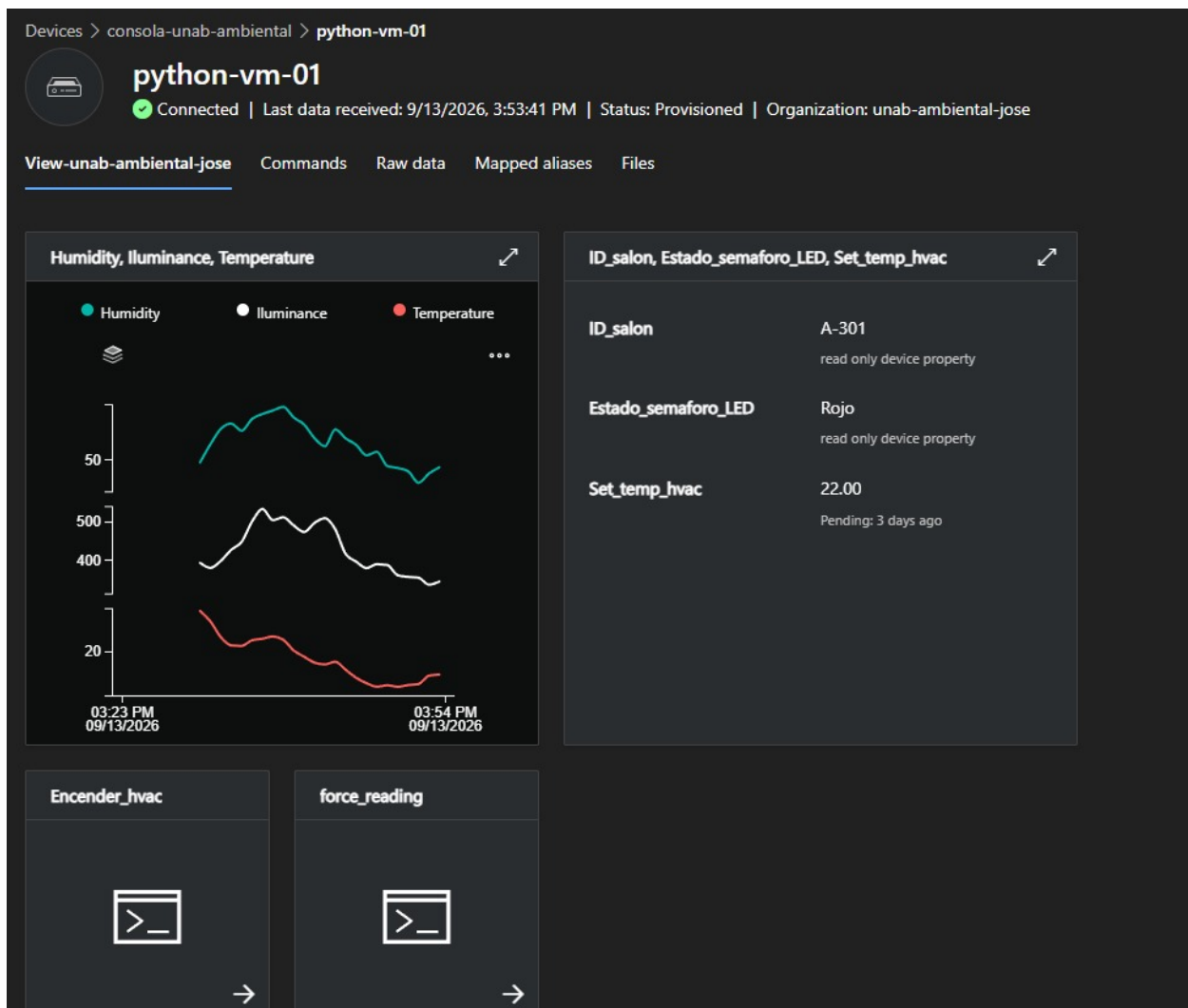


Figura 2: Dispositivo `python-vm-01` conectado, telemetría en vivo y propiedades del gemelo (`ID_salon`, `Estado_semaforo_LED`, `Set_temp_hvac`).

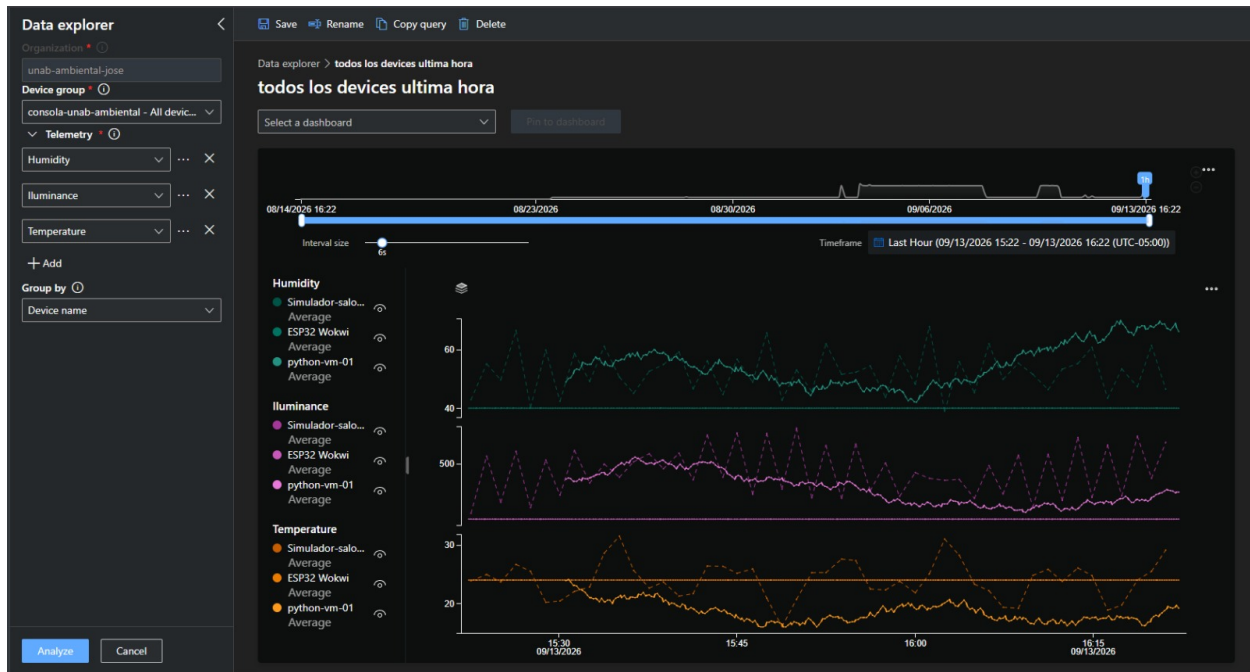


Figura 3: Data explorer con los tres dispositivos de la flota publicando simultáneamente (Simulador nativo, ESP32 Wokwi y python-vm-01) – evidencia de que el Lab 2 sigue vivo.

Nota sobre Cloud-to-Device (C2D)

Durante la Etapa 3 se intentó capturar evidencia de un mensaje Cloud-to-Device clásico. La API REST (`POST /api/devices/{id}/sendMessage`) respondió **404 Not Found: API not found**, y la interfaz actual de IoT Central ya no ofrece la opción "Send message to device.^{en} el menú del dispositivo – es un retiro funcional de la plataforma, no un error del laboratorio. Como evidencia de canal descendente (downlink) real y funcional se documentan en su lugar: (a) las suscripciones concedidas por SUBACK al topic `devices/23z8rpgm6s4/messages/devicebound/#`; (b) los métodos directos ($4\times$ *force_reading* respondidos con 200 por el cliente sin SDK, ver Figura anterior); y (c) la lectura del gemelo por MQTT puro (`GET $iothub/twin/GET`, respuesta 200 con *reported/desired*).

6. Repositorio

Todo el código (SDK del Lab 2, cliente MQTT explícito, proyecto Wokwi, bitácora y evidencias) está versionado en: https://github.com/joserraldo/Iot_learning

7. Lecciones aprendidas

- Una primera tanda de mediciones se corrió por error contra una app de IoT Central distinta a la del proyecto (credenciales cruzadas en el `.env` de la VM); se detectó por inconsistencia de *hub/id scope*, se corrigió, y se repitió la batería completa de mediciones contra la app correcta antes de reportar cualquier número en este informe.
- Reproducir a mano el token SAS y los topics del SDK confirma que no hay magia oculta: es MQTT 3.1.1 estándar sobre TLS, con autenticación por firma HMAC-SHA256 de la *primary key*.

- El hallazgo más relevante de este laboratorio es la degradación silenciosa de QoS 2 a QoS 1 por parte de IoT Hub/Central – algo que el SDK nunca expone porque simplemente no lo pide.

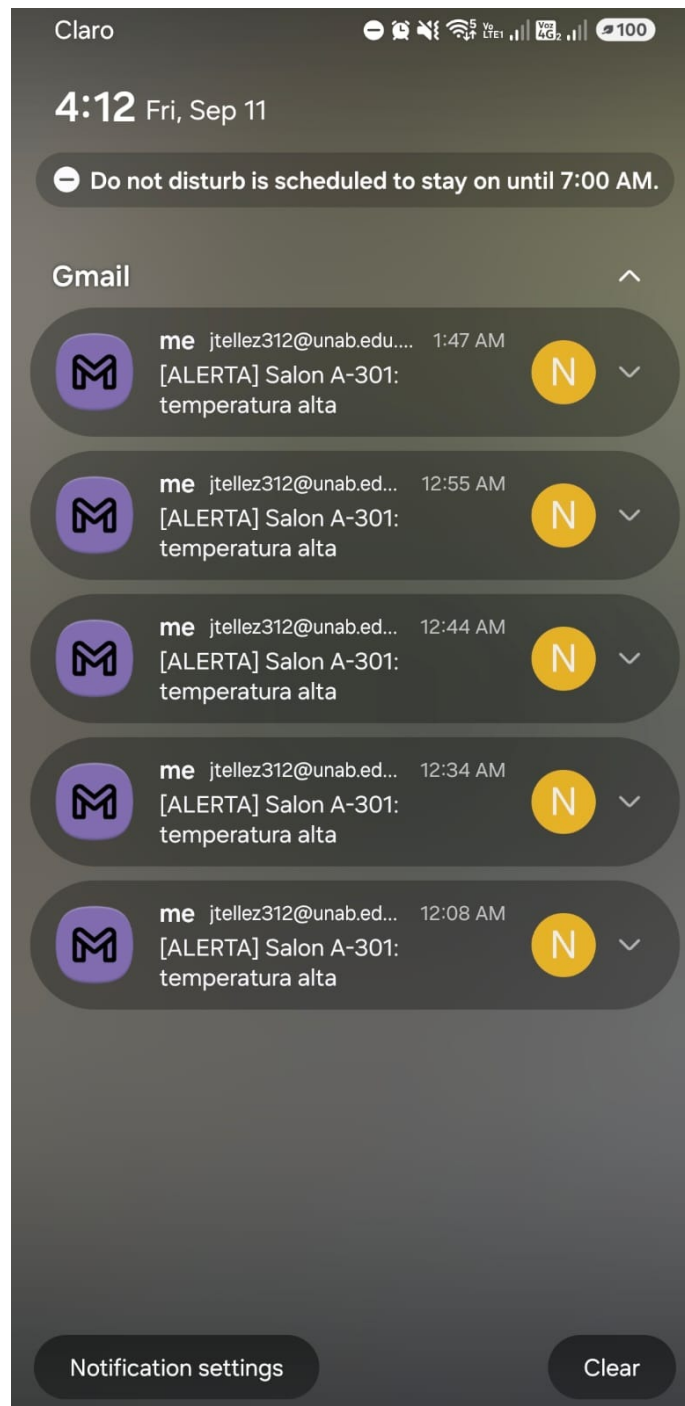


Figura 4: Alertas de correo de la regla *Temperature* >27 recibidas mediante el nuevo servicio de envío de correo (Brevo), añadido tras el cierre del Lab 2.

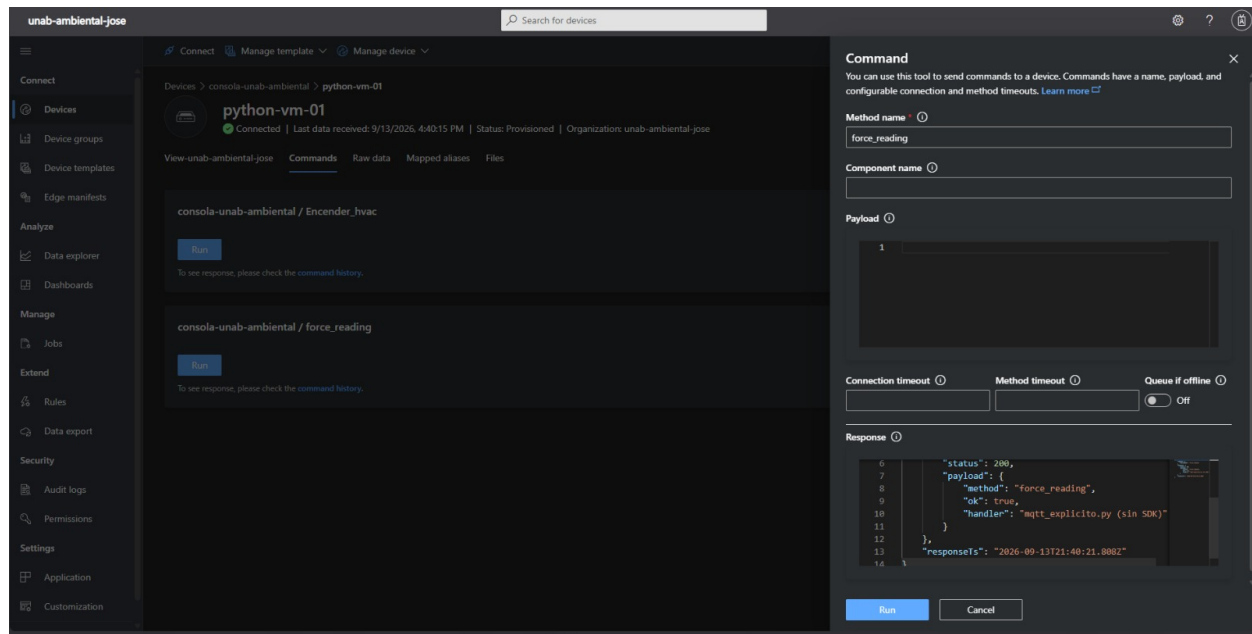


Figura 5: Comando *force_reading* ejecutado desde el portal y respondido con estado 200 por el cliente MQTT explícito (handler: "mqtt_explicito.py (sin SDK)"), sin pasar por el SDK.

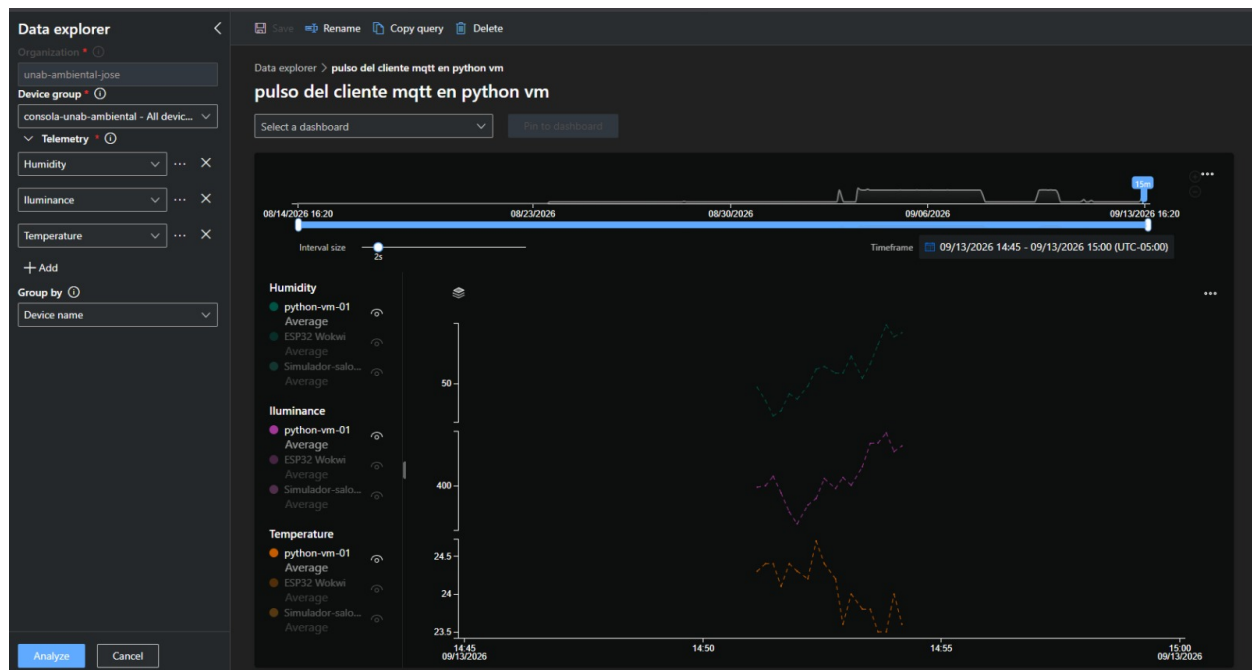


Figura 6: Pulsos de telemetría del cliente MQTT explícito sobre *python-vm-01* en la franja 14:45–15:00 (hora local), correspondiente a la re-medicación formal de la Etapa 3.